MiTRAA TaskFlow: API Reference (v3)

Oct 31, 2026 · @Deenesh

The MiTRAA TaskFlow web app (called MyMitraa in earlier versions of this document) is powered by 102 REST endpoints across 7 controllers, listed here module by module in plain language so developers, testers and analysts can all use them. Version 2 checks every endpoint against the six controllers shared on Oct 31, 2026, and adds what the redesigned screens need from the API.

What changed in this version

Area	Change
Checked	All 102 endpoints match the controllers: 24 in Task Tracker, 31 in Global File Uploads, 13 each in Weekly Update and Dropdown Management, 10 each in Meeting and Leave, 1 in Task Owner Email. 21 declare the Authorization header as required. Nothing in version 1 needed correcting.
Screen map	Now covers 8 screens. The old Dashboard is split into the new Home Dashboard and Reports (the charts screen).
Values	New table of the values the screens show (status, priority, team, leave type, dates) and what the front end must map.
Proposed additions	New list of calls the redesigned Home Dashboard needs that do not exist yet. These are suggestions, not built endpoints.
Developer notes	Four new notes, including a duplicate commented-out endpoint and the missing upload call on Dropdown Management.

At a glance

Global File Uploads (31) and Task Tracker (24) hold more than half of the endpoints; the other five modules share one simple pattern.

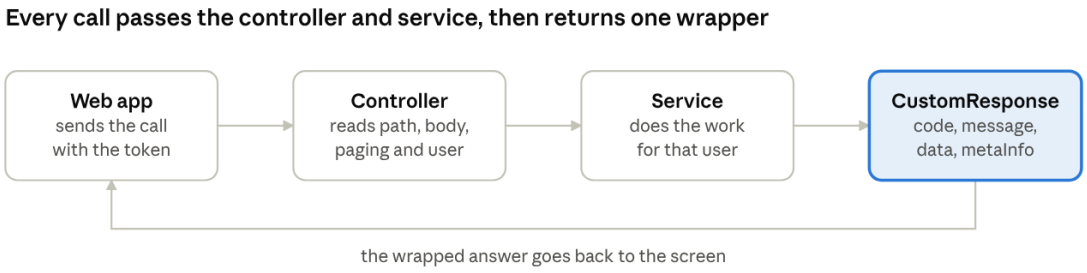
Module	Screen it serves	Base path	Endpoints
Task Tracker (Feedback)	Task Tracker, Home Dashboard, Reports, View Documents, Calendar View	/api/feedback	24
Meeting Time Tracker	Meeting Tracker, Home Dashboard, Reports (meeting time)	/api/ meeting_time_tracker	10
Leave / WFH Tracker	Leaves / WFH Tracker, Home Dashboard, Reports (leave view)	/api/ optima_leave_tracker	10

Module	Screen it serves	Base path	Endpoints
Weekly Update Tracker	Project Status	/api/ optima_weekly_update_ tracker	13
Dropdown Management	Dropdown Management	/api/ dropdown_management	13
Task Owner Email	Task owner emails	/api/task_owner_email	1
Global File Uploads	No screen in the shared captures	/api/global_uploads	31
Total			**102**

The Screen column is matched by name, not by reading the front-end code; see the screen-to-API map near the end.

How every call works

Every call is made by a signed-in user and comes back in the same wrapper, so one client can handle all 102 endpoints the same way.



request flow · 4 steps

The controller reads the request and hands it to a service; the answer always leaves in the wrapper.

Rule	What it means
Sign-in	The signed-in user is passed to every method. Send the `Authorization` token on every call; 21 endpoints also declare it as a required header and fail without it.
Response wrapper	Most calls return `CustomResponse`: a success code, a success message, the data, and `metaInfo` holding the service name.
Exceptions to the wrapper	The five `get_parameter_list` calls return the search-field list directly. The three Global File Uploads upload calls return whatever the service returns.

Rule	What it means
Search body	Search and list calls (PUT) send a `SearchVM` JSON body with the filters. Its fields live in the `SearchVM` class, which was not part of the files shared.
Paging	Calls that take `Pageable` accept the standard Spring paging parameters: page, size and sort.
Verbs	GET reads one item or a simple list, POST creates or uploads, PUT updates and also runs searches, dashboards and downloads, DELETE removes.
Body formats	JSON for most calls. Form-data for feedback save/edit/sub-task calls and the weekly-update save/update/upload calls. Multipart for file uploads.

Standard tracker endpoints

Meeting Tracker, Leaves/WFH, Weekly Update and Dropdown Management all expose the same 9 calls; learn them once and you know 36 of the 102 endpoints. Paths below are added to each module's base path.

Action	Verb	Path	What it does	Sends
Save	POST	/save	Creates a record	Record as JSON
Get one	GET	/get/{id}	Fetches one record	Record id in the path
Get all	PUT	/getAll	Returns every record that matches the search	Search body (`SearchVM`)
Get a page	PUT	/get_with_pagination	Same search, one page at a time	`SearchVM` + paging
Update	PUT	/update/{id}	Changes a record	Record id + changes as JSON
Download	PUT	/download	Prepares an export of the records	Filter as JSON + `Authorization`
Search fields	GET	/get_parameter_list	Lists the fields advanced search can use (`AdvanceSearchList`)	`Authorization`
Advanced search	PUT	/advance_search	Filtered, paged search	`SearchVM` + paging + `Authorization`
Delete	DELETE	/delete/{id}	Removes a record	Record id in the path

Action	Verb	Path	What it does	Sends

How each module differs from that pattern:

Module	Base path	Save takes	Update takes
Meeting Time Tracker	/api/ meeting_time_tracker	`MeetingTimeTracker` (JSON)	`MeetingTimeTrackerVM` (JSON)
Leave / WFH Tracker	/api/ optima_leave_tracker	`OptimaLeaveTracker` (JSON)	`OptimaLeaveTrackerVM` (JSON)
Dropdown Management	/api/ dropdown_management	`DropdownManagement` (JSON)	`DropdownManagementVM` (JSON)
Weekly Update Tracker	/api/ optima_weekly_update_tracker	`OptimaWeeklyUpdateTrackerVM` (form-data)	Same form-data, sent with POST /update and no id in the path

Extra endpoints per module

On top of the standard 9, these 11 calls are specific to one module. Paths are shown in full.

Module	Verb	Full path	What it does	Sends
Meeting Tracker	PUT	/api/ meeting_time_tracker/ dashboard	Numbers for the meeting-time charts	`MeetingTimeTrackerVM`
Leaves / WFH	PUT	/api/ optima_leave_tracker/ dashboard	Numbers for the leave and WFH summary	`OptimaLeaveTrackerVM`
Project Status	PUT	/api/ optima_weekly_update_tracker/dashboard	Numbers for the weekly-status dashboard	`OptimaWeeklyUpdateTrackerVM`
Project Status	POST	/api/ optima_weekly_update_tracker/uploadfile/{id}	Attaches documents to a weekly update	Record id + `DocumentUploadVM` (form-data)
Project Status	GET	/api/ optima_weekly_update_tracker/get_fetch_all/ {projectName}	Fetches the weekly data for one project; the only call here that does not take the signed-in user	Project name in the path
Project Status	PUT	/api/	Sends the weekly report	`MailRequestVM`

Module	Verb	Full path	What it does	Sends
		optima_weekly_update_tracker/send-weekly-mail	by email	
Dropdown Management	PUT	/api/dropdown_management/dropdown	Returns the options for a dropdown	`DropdownManagement`
Dropdown Management	PUT	/api/dropdown_management/dropdown-subvalue	Returns the sub-value options, one level deeper	`DropdownManagement`
Dropdown Management	PUT	/api/dropdown_management/dropdown-subvalue-module	Returns the options one level deeper again	`DropdownManagement`
Dropdown Management	PUT	/api/dropdown_management/dropdown-subvalue-module-submodule	Returns the deepest level of options	`DropdownManagement`
Task Owner Email	POST	/api/task_owner_email/save	Saves a task owner's email record; this is the module's only call	`TaskOwnerEmail` (JSON)

The four dropdown calls are named after the level they serve (value, sub-value, module, sub-module); the exact level behaviour sits in the service, which was not shared.

Task Tracker (Feedback)

The Task Tracker module has 24 endpoints under `/api/feedback`; they cover tasks, sub-tasks (called epics in the paths), files and effort. Paths are added to the base path.

Job	Verb	Path	What it does	Sends
Create and edit	POST	/save_feedback	Creates a task	`FeedbackHistoryVM` (form-data)
Create and edit	POST	/edit_feedback	Edits a task	`FeedbackHistoryVM` (form-data)
Create and edit	POST	/assign_task	Assigns a task to someone	`AssignTaskVM` (JSON)
Create and edit	POST	/close_task	Closes a task	`FeedbackHistoryVM` (JSON)
Create and	DELETE	/	Deletes a task	Task id in the path

Job	Verb	Path	What it does	Sends
edit		get_feedback_by_id /{feedback_id}		
Read and search	GET	/ get_feedback_by_id /{feedback_id}	Fetches one task	Task id in the path
Read and search	PUT	/get_all_feedback	Paged list of tasks	`SearchVM` + paging
Read and search	GET	/ feedback_numerical _count	Task counts for summary figures	Nothing
Read and search	GET	/ get_all_ticket_numb ers	Lists all ticket numbers	`Authorization`
Read and search	GET	/get_parameter_list	Lists the fields advanced search can use	`Authorization`
Read and search	PUT	/advance_search	Filtered, paged search of tasks	`SearchVM` + paging + `Authorization`
Read and search	PUT	/ subtask_advance_se arch	Filtered, paged search of sub-tasks	`SearchVM` + paging + `Authorization`
Read and search	GET	/searchable-fields	Lists the sub-task fields that can be searched	`Authorization`
Read and search	PUT	/get_status_record	Paged status records	`SearchVM` + paging
Sub-tasks	POST	/add_subtask	Adds a sub-task to a task	`FeedbackHistoryVM` (form-data)
Sub-tasks	POST	/update_subtask/ {feedbackHistoryId}/ {epicId}	Edits a sub-task	`EpicVM` (form-data)
Sub-tasks	DELETE	/delete_subtask/ {feedbackHistoryId}/ {epicId}	Deletes a sub-task	Both ids in the path
Files	POST	/upload	Uploads a file	File (multipart) + `Authorization`
Files	POST	/upload/	Uploads a file against a	Ticket number + file +

Job	Verb	Path	What it does	Sends
		{ticketNumber}	ticket	`Authorization`
Files	DELETE	/delete_file/{ticketNumber}	Deletes the file of a ticket	Ticket number in the path
Files	PUT	/get_uploaded_files	Paged list of uploaded files	`SearchVM` (date range) + paging
Effort	PUT	/daily_task_tracker	Daily effort records	`EffortRecordVM`
Effort	PUT	/dashboard	Numbers for the effort and status charts (Reports screen)	`EffortRecordVM`
Effort	PUT	/add_worklog_hours	Adds logged hours to a task or sub-task	Query `feedbackHistoryId`, optional `epicId`, body list of `WorkLogEntry`

Global File Uploads

The largest module has 31 endpoints under `/api/global_uploads`; they upload data files, review them, process them in batches and clean up afterwards. None of the captured screens clearly matches it. Paths are added to the base path; `operationDateTime` is a date sent as a query parameter.

Job	Verb	Path	What it does	Sends
Upload	POST	/upload_file	Uploads a data file	File + `category` + `operationDateTime`
Upload	POST	/automatic_upload_file	Uploads a file for automatic processing	File + `category` + `operationDateTime`
Upload	POST	/upload_templete_manual_file	Uploads a template or manual file	File + `category` + `startDate`
Upload	POST	/MechanicalIntegrity_upload_file	Uploads a Mechanical Integrity file	File + `category` + `inspectionDate` (yyyy-MM-dd) + `description`
Upload	POST	/update_by_id/{id}	Replaces a template file's details	`GlobalFileUploadVM` (form-data) +

Job	Verb	Path	What it does	Sends
				`Authorization`
Lists and lookups	PUT	/get_uploaded_files	Paged list of uploaded files	`SearchVM` + paging
Lists and lookups	PUT	/get_MechanicalIntegrity_uploaded_files	Paged list of Mechanical Integrity files	`SearchVM` + paging
Lists and lookups	GET	/get_import_fileNames	Lists import file names	Nothing
Lists and lookups	GET	/get_latestDate	Returns the latest date held	Nothing
Lists and lookups	GET	/get_latest_approved_date	Returns the latest approved date	Nothing
Lists and lookups	GET	/get_global_import_dropdown_list	Returns the import dropdown list	`operationDateTime`
Lists and lookups	GET	/get_global_numerical_analysis	Returns the FTP numerical analysis	Nothing
Lists and lookups	POST	/save_downloaded_count	Records how many files a user downloaded	`Authorization` + optional `FilesDownloadedCountDomain`
Review	GET	/update_approved_by_id/{globalFileUploadsId}	Approves an uploaded file	File record id
Review	GET	/reject_by_id/{globalFileUploadsId}	Rejects an uploaded file	File record id
Review	DELETE	/delete_by_id/{globalFileUploadsId}	Deletes a template file	File record id
Review	GET	/validate_all_files	Validates all files for a date	`operationDateTime`
Review	PUT	/restriction	Stops duplicate import files	`SearchVM`

Job	Verb	Path	What it does	Sends
			being loaded	
Batch processing	GET	/chron_processed	Runs the cron processing for all tables	Nothing
Batch processing	GET	/chron_processed_date	Same, for one operation date	`operationDateTime`
Batch processing	GET	/chron_processed_date_without_vfm	Same, leaving out VFM	`operationDateTime` (text date)
Batch processing	GET	/chron_processed_for_extra_table	Cron processing for the Sand monitoring and ESP diagnostic tables	Nothing
Batch processing	PUT	/automatic_process	Runs the automatic cron file processing	`SearchVM`
Batch processing	PUT	/automatic_process_without_import	Same, without import files; built for the mobile app group process	`SearchVM`
Batch processing	PUT	/create_excel	Creates an Excel file	`SearchVM`
Batch processing	PUT	/update_import12	Updates the plant Import 12 data	`SearchVM`
Batch processing	GET	/update_table	Refreshes the table data	Nothing
Clean-up	GET	/remove_above_data	Removes loaded data from a date onward	`operationDateTime`
Clean-up	PUT	/remove_above_data	Same, driven by a search body	`SearchVM`
Clean-up	GET	/remove_above_data_with_date	Same, with its own date handling	`operationDateTime`
Clean-up	GET	/remove_above_data_otherthan_import_data	Removes everything except import data	`operationDateTime`

Data fields by module

These are the fields each record holds, taken from the domain classes. Every record also carries the common BaseDomain fields (table below). Names are shown as stored in the database; the JSON name in requests is normally the Java name, so ticket_number is ticketNumber (confirm against a real response).

Common fields on every record (BaseDomain)

The system fills these in; the front end does not send them, except that it can read them.

Field (as stored)	Type	What it is	Note
created_on, created_by	Date, text	Who made the record and when	Filled automatically
modified_date_time, modified_by	Date, text	Last change	Filled automatically
active_flag	true/false	Record is active	Default true
delete_flag, delete_on	true/false, date	Soft delete	Deleted records stay in the database
status_flag	Text	Record state	Default is the constant Constants.ACTIVE; stored as status_flag
history_flag	true/false	Record is a history copy	Default false
report_generated_date	Date	When a report was made from it	
Timeline View	Tasks on a time line; user taken from the token	Task Tracker: advance_search or get_all_feedback	Likely: you confirmed it

Work log entry (inside a Task Tracker record)

Each task keeps a list of daily work entries.

Field (as stored)	Type	What it is	Note
date	Date	Day worked	
hours	Number	Hours spent	The service adds each entry's hours to actual_effort
description	Text	What was done	

Task Tracker record (FeedbackHistory, collection feedback)

One record is a task or a sub-task. It is the record behind Calendar View, Timeline View, View Documents and the Reports charts.

Field (as stored)	Type	What it is	Note
ticket_number	Text	Ticket no.	Shown as the ticket ID; used to search
subject / description / task_description	Text	Title and details	Subject is the one-line title
status, priority	Text	Dropdown	Values come from Dropdown Management
project, module, sub_module, epic, customer, instance, product, project_category	Text	Dropdown	Cascading: project then module then sub-module; epic = parent of a sub-task
team, task_type, classification, category, category_description	Text	Dropdown / text	Grouping fields for reports
task_owner, assigned_by, who_reported, reviewed_by	Text	Person	Owner = person doing the task; calendar and timeline filter on this
estimated_start_date, estimated_close_date	Date	Planned dates	Bar start and end on Calendar View and Timeline View
actual_start_date, actual_close_date, date_assigned	Date	Real dates	Set while the task runs
estimated_effort, actual_effort, productivity_savings	Number (hours)	Effort	actual_effort grows as work-log hours are added; do not make it editable by hand. Feeds the effort charts on Reports
coding, ai_tool, internal_meeting	Number (hours)	Effort split	Hours per kind of work
comments, reviewer_comments	Text	Notes	
work_log_entries	List of WorkLogEntry	Daily log	Each entry: date, hours, description (table below)
attachments, file_name, uploaded	List of text, text, true/false	Files	Feed View Documents
subtask_count, last_subtask_update	Whole number, date	Sub-task roll-up	Read-only counters

Meeting record (MeetingTimeTracker)

Fields the Add New and Edit forms on Meeting Tracker need.

Field (as stored)	Type	What it is	Note
date	Date	Meeting day	Required for the calendar and search
customer, project, module, team, category	Text	Dropdown	Values from Dropdown Management
meeting_by	Text	Organiser	
meeting_with	List of text	Attendees	Multi-select
lead_attended, architect_attended	true/false	Tick boxes	
start_time, end_time	Date-time	Time range	
total_time	Number	Duration	Probably calculated from the two times; confirm
comments	Text	Notes	

Leave / WFH record (OptimaLeaveTracker)

Fields the Add New and Edit forms on Leaves / WFH need.

Field (as stored)	Type	What it is	Note
name	Text	Person	Taken from the token unless an admin adds for someone else (to confirm)
role	Text	Job role	
start_date, end_date	Date	Range	Drawn as bars across the month calendar
day	Text	Full day or half day	Exact values not supplied
leave_type	Text	Leave, WFH or Holiday	Exact values not supplied; also drives colour
comments	Text	Notes	

Weekly Update record (OptimaWeeklyUpdateTracker)

One record is one project for one week; this is what the Project Status grid shows.

Field (as stored)	Type	What it is	Note
project_name, customer_name, customer_geography, industry	Text	Project identity	Row labels on Project Status
state (as status), current_state, current_status, project_status, last_week_status	Text	Status	Note: the API field called status is stored as state
priority, risk, project_summery	Text	Priority, risk, summary	Spelling project_summery is as in the code
start_date, go_live_date	Date	Project dates	
weekly_start_date, weekly_end_date	Date	Week covered	One record per project per week
ref, code_base, documentation, kt_provider	Text	Links and references	
hld_link, lld_link	Text	Design document links	
hld_attachment, lld_attachment	List of text	Design files	Uploaded with uploadfile
testers, fe_developers, be_developers, python_developers, ux_designers, shared_resources	List of text	People per role	Show as chips; the screen shows row labels only
totalEnv	Whole number	Number of environments	
dev_url, qa_url, la_url, other_url	Text	Environment links	
dev_arrow, qa_arrow, la_arrow, other_arrow	true/false	Arrow shown or hidden	Default true; these are the blue arrows seen on the screen

File upload record (GlobalFileUploads)

One record is one uploaded file.

Field (as stored)	Type	What it is	Note
category, source, description	Text	What the file is	
file_name, file_path	Text	Stored file	
operation_date_time, processed_date_time	Date-time	Upload and process times	
isProcessed, isApproved	true/false	Review state	Set by the process and approve calls
index	Number	Order in a batch	
start_date, end_date, inspection_date	Date	Data period and inspection day	inspection_date added 14 May 2025

Calendar View and Timeline View

Both screens read the Task Tracker, and the signed-in user comes from the token, so the front end does not send a user name. Use get_all_feedback for the plain list or advance_search when filters are needed (SearchVM). The estimated start and close dates are the bar ends; the task owner, project, status and priority are the filters. Timeline View groups the same bars by owner or by project.

Screen to API map

Most screens map cleanly to one module; the Match column says how sure we are, because the links come from names and not from the front-end code. Home Dashboard is new in version 2, so its rows say what it should call, not what the front end calls today.

Screen	Calls it needs	Module	Match
Home Dashboard: KPI cards and progress ring	Counts per status	Task Tracker: feedback_numerical_count	Likely
Home Dashboard: My Tasks	Tasks of the signed-in user by Today, This Week, Overdue	Task Tracker: advance_search or get_all_feedback (SearchVM fields not shared)	Guess
Home Dashboard: Upcoming Schedule	Leave, WFH and holidays for the coming days	Leave / WFH Tracker: get_with_pagination or getAll with a date range	Likely
Home Dashboard: upcoming meetings	Meetings after today	Meeting Time Tracker: getAll with a date range (future meetings may not be stored)	Guess

Screen	Calls it needs	Module	Match
Home Dashboard: people not filling tasks	Names of people with no tasks	Task Tracker: dashboard, as on Reports	Guess
Meeting Tracker	Search, page, download, add, edit, delete	Meeting Time Tracker, standard 9	Clear
Leaves / WFH Tracker	Month calendar of entries; add, edit, delete	Leave / WFH Tracker, standard 9	Clear
Reports: Leaves / WFH view	Pie and summary numbers	/api/optima_leave_tracker/ dashboard	Clear
Reports: Meeting time view	Minutes per person per project	/api/meeting_time_tracker/ dashboard	Clear
Reports: Status and Effort views	Work-item counts, effort hours	/api/feedback/dashboard, /daily_task_tracker, /feedback_numerical_count	Likely
Project Status	Weekly grid per project, attachments, weekly email	Weekly Update Tracker: get_fetch_all, dashboard, uploadfile, send-weekly-mail	Clear
View Documents	File list, upload, delete, ticket search	Task Tracker: get_uploaded_files, upload, delete_file, get_all_ticket_numbers	Likely
Calendar View	Tasks drawn as bars across dates	Task Tracker: advance_search or get_all_feedback	Guess
Dropdown Management	List, add, edit, delete owners; cascading options	Dropdown Management, standard 9 + 4 dropdown calls (no upload call)	Clear
No screen seen	Task owner email save	Task Owner Email	Guess: may belong to Dropdown Management's Task Owner
No screen seen	File import, review, batch processing	Global File Uploads	None in the captures

Values the screens show

The screens write values in plain wording, and the API may use different strings. The strings below are as they appear on screen; confirm each against the service code before mapping, and keep the mapping in one shared place in the front end rather than per screen.

Field	Seen on screen	Note for the front end
Task status	Closed, In_Progress, Open; On Hold appears as a choice on Calendar View	The Status Overview legend prints In_Progress with an underscore. The redesign shows In Progress; map it once.
Priority	Critical - P0, Urgent - P1, Normal - P2, Low - P3	Shown on the Priority Breakdown chart; Low - P3 is cut off in the capture.
Task team	QA, Project Management, Mobile Development, BE Development, FE Development; the effort chart also lists IoT, UI / UX Design, Architecture and Python Development	Use one list for filters and charts.
Meeting module	All Module, All Modules, Project Requirements, Internal Meeting	All Module and All Modules mean the same thing. One value should win; clean the data or map it.
Leave type	Planned Leave, Unplanned Leave, WFH (Approved), WFH (Unapproved), Holiday	The calendar bar reads Name (Type); the legend calls planned leave simply Leave.
Dates	Tables: 03-Sep-2026. Ranges: 10/01/2026 - 10/31/2026. Dropdown Management: 09-01-2026	The API takes Date without a format on most Global File Uploads calls, text on one, yyyy-MM-dd on Mechanical Integrity. Convert in one date helper at the edge; the screens show DD-Mon-YYYY.
Filters on Reports	Date Type (Actual Date), Duration (Month), Month Range	These live in SearchVM or EffortRecordVM, neither of which was shared.
Time	Effort in hours with two decimals (2975.78 hrs); meeting time axis in minutes, labelled 52h or 2h 45m	Format minutes as hours and minutes in the UI.

Proposed additions (not built)

These calls do not exist in the 102 endpoints. They are suggestions to give the backend team a starting point, driven by the Home Dashboard; names and shapes are theirs to decide. No existing endpoint needs to change.

Proposed call	Why	Suggested content
GET /api/feedback/dashboard_summary	KPI cards, progress ring, trend arrows	For the signed-in user and a chosen period: total, open, in progress, closed, on hold, overdue, due today, due this week, and the same counts for the previous period.
PUT /api/feedback/my_tasks	My Tasks panel and its tab counts	Paged tasks of the signed-in user for the tabs All, Today, This Week and Overdue, with ticket number, title, team, priority, due date and owner. May be unnecessary if advance_search supports owner and due-date filters.
GET /api/optima_leave_tracker/upcoming	Upcoming Schedule	Leave, WFH and holiday entries for the next N days.
GET /api/meeting_time_tracker/upcoming	Upcoming Schedule	Only if future meetings are stored.
GET /api/notifications and PUT /api/notifications/{id}/read	Notification bell	List with an unread count; mark one as read.
GET /api/users/me	User menu	Name, role and a photo if one exists. Change Password and Sign out belong to the authentication controller, which was not shared.
POST /api/dropdown_management/upload	Second arrow on Dropdown Management	Only if that arrow is an upload; otherwise remove the arrow.

Developer notes

Ten things in the controllers are worth a second look; none stops the APIs working today. The first six are from version 1, the last four are new.

Note	Where	Why it matters
GET calls that change data	Global File Uploads: `update_approved_by_id`, `reject_by_id`, `remove_above_data*`, `chron_processed*`, `update_table`,	A GET that approves, rejects or deletes can be fired by a link or a browser prefetch; POST, PUT or DELETE is safer.

Note	Where	Why it matters
	`validate_all_files`	
Delete named "get"	`DELETE /api/feedback/get_feedback_by_id/{feedback_id}`	The path reads like a read call; it deletes the task.
Two `SearchVM` classes	`api_vm.SearchVM` in Dropdown, Leave, Weekly and Meeting; `web.rest.vm.SearchVM` in Task Tracker and Global File Uploads	Confirm both accept the same filter fields, or the same JSON may behave differently per module.
File parameter style	`@RequestBody MultipartFile` on 6 upload calls (2 in Task Tracker, 4 in Global File Uploads)	Spring's usual choice is `@RequestParam` or `@RequestPart`; confirm how the front end sends the file and under which field name.
Date inputs differ	Global File Uploads takes `Date` without a format on most calls, a text date on one, and `yyyy-MM-dd` on Mechanical Integrity	The caller must match each format; one shared format would be simpler.
Small inconsistencies	Weekly update uses `POST /update` with no id while the others use `PUT /update/{id}`; the four dropdown calls log "updated Equipments details"; Task Tracker reuses `DAILY_TASK_TRACKER_FETCHED_SUCCESSFULLY` for non-daily calls	Harmless to users, but confusing in logs and for new developers.
Duplicate endpoint in source	Global File Uploads: get_global_import_dropdown_list appears twice, once commented out	Only the live copy counts (31 endpoints). Delete the commented one so nobody counts 32.
No upload on Dropdown Management	DropdownManagementController	The screen has two blue arrows, probably download and upload, but only download has a call.
Status written with an underscore	Reports charts show In_Progress	Agree one canonical status value and map it to a label once in the front end.
Home Dashboard needs several	Task Tracker, Leave / WFH Tracker	No single call serves the

Note	Where	Why it matters
calls	and Meeting Time Tracker	page. Load the panels in parallel and let each fail on its own so one error never blanks the page.
Task Tracker and Weekly Update records declare their own status field while BaseDomain already has one	FeedbackHistory, OptimaWeeklyUpdateTracker	The child field hides the parent field (BaseDomain stores status_flag), so JSON and queries can pick the wrong one. Rename one of them.
Weekly record: status is stored as state	OptimaWeeklyUpdateTracker	The Java field is status but the stored name is state, so queries must use state.
Weekly record: python_developers, ux_designers and shared_resources are in the class but missing from toString in two places	OptimaWeeklyUpdateTracker	Harmless, but logs hide these fields.
Upload record: isProcessed and isApproved are not annotated	GlobalFileUploads	They are stored as isProcessed and isApproved, unlike the snake_case of every other field.
Meeting record: total_time is a stored number	MeetingTimeTracker	Nothing shows it is calculated from start and end, so the two can disagree.